Descripción Ver entrega

Límite de entrega: viernes, 25 de abril de 2025, 10:30
Ficheros requeridos: prueba.py, supermarket.py (Descargar)
Tipo de trabajo: Individual

Gestión de Cola en un Supermercado

Se pide finalizar el desarrollo de la clase `CheckoutQueue` del módulo `supermarket`. Cada objeto de dicha clase representa la **cola** de clientes de una caja en un supermercado. Se proporciona la clase `Client` completada; y la clase `CheckoutQueue` por completar que ya incluye un constructor básico y una clase interna `Node` a usar para los **nodos** de la lista encadenada conteniendo los clientes de la cola.

Cada objeto de la clase `Client` representa a un **cliente** empleando dos atributos públicos: `name` para el nombre y `total_cost` para el total a pagar por su compra.

Para completar la clase `CheckoutQueue` deberás desarrollar los siguientes métodos y propiedades (pudiendo para ello añadir o modificar sus actuales atributos o métodos si fuera necesario):

Métodos y propiedades a desarrollar:

1. **Método** `enqueue`
   Representa la llegada de un cliente (recibido por parámetro) a la cola, añadiéndolo por el final.

2. **Método** `dequeue`
   Representa el que un cliente ha pasado por caja. Para ello, quita el cliente que estaba al principio de la cola y lo devuelve. Además, ha de actualizar apropiadamente lo ingresado en esa caja desde que se abrió su cola. Devuelve `None` si no había cliente en la cola.

3. **Propiedad** `daily_sales`
   Esta propiedad de solo lectura nos devuelve lo ingresado en esa caja desde que se abrió la cola.

4. **Método** `split`
   Este método está pensado para que los clientes de la cola pasen a ser atendidos en dos cajas. Para ello crea una nueva cola y la devuelve tras repartir los clientes, de forma que:

   - La **cola original** conservará los clientes de posiciones **pares** (0, 2, 4, ...),
   - La **nueva cola** contendrá los clientes de posiciones **impares** (1, 3, 5, ...),
   - Manteniendo en ambas el orden relativo original de sus clientes.

Recomendaciones:

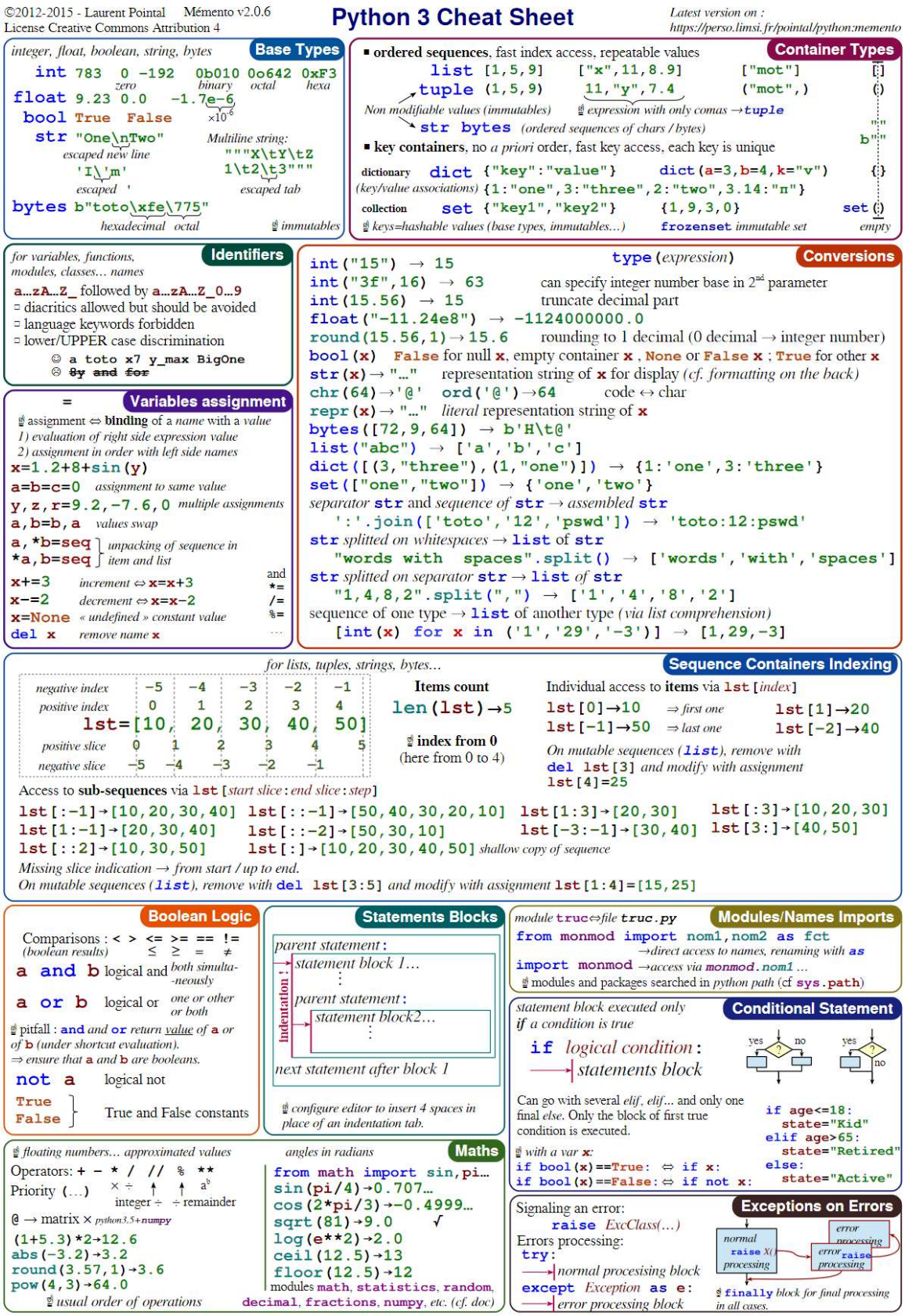
- **Localice los errores** en su código modificando y ejecutando el fichero `prueba.py`. El contenido de este fichero no se testea, pero si contiene errores de compilación puede impedir la ejecución de las pruebas.
- La clase `CheckoutQueue` hereda de la clase `ContainerBase`, cuyo método `__str__` nos facilita la visualización de la cola.
- **Despliegue** la pestaña Descripción justo **antes** de pulsar Evaluar.
- **Despliegue** la pestaña **Comentarios aun cuando se muestre** algún mensaje en la de **Compilación** ya que en comentarios aparecerán el resultado de las pruebas realizadas.

Ejemplo de uso:

```
cola = CheckoutQueue()
print(cola) # ➡️None⬅️
cola.enqueue(Client("Ana", 35))
cola.enqueue(Client("Luis", 50))
cola.enqueue(Client("Marta", 25))
print(cola) # ➡️[Ana: 35]->[Luis: 50]->[Marta: 25]⬅️->None
print(cola.dequeue()) # Ana: 35
print(cola.dequeue()) # Luis: 50
print(cola.daily_sales) # 85
print(cola) # ➡️[Marta: 25]⬅️->None
```

?

Nota: las flechas señalan los nodos referenciados por los atributos `first` y `last`, mientras que `->` representa a `next`.



Conditional Loop Statement

statements block executed ***as long as*** *condition is true*

```
while logical condition:
    statements block
```

```
s = 0
i = 1
while i <= 100:
    s = s + i**2
    i = i + 1
print("sum:",s)
```

initializations ***before*** *the loop condition with a least one variable value (here* `i`*)*

make condition variable change !

Algo: $s = \sum_{i=1}^{i=100} i^2$

beware of infinite loops!

Loop Control

`break`	*immediate exit*
`continue`	*next iteration*

`else` *block for* ***normal*** *loop exit.*

Iterative Loop Statement

statements block executed ***for each*** *item of a container or iterator*

```
for var in sequence:
    statements block
```

Go over sequence's **values**

```
s = "Some text"
cnt = 0
for c in s:
    if c == "e":
        cnt = cnt + 1
print("found",cnt,"'e'")
```

initializations ***before*** *the loop*

loop variable, ***assignment managed by*** `for` *statement*

Algo: count number of `e` *in the string.*

loop on dict/set ⇔ loop on keys sequences

use *slices* to loop on a subset of a sequence

Go over sequence's **index**

- modify item at index
- access items around index (before / after)

```
lst = [11,18,9,12,23,4,17]
lost = []
for idx in range(len(lst)):
    val = lst[idx]
    if val > 15:
        lost.append(val)
        lst[idx] = 15
print("modif:",lst,"-lost:",lost)
```

Algo: limit values greater than 15, memorizing of lost values.

Go simultaneously over sequence's **index** and **values**:

```
for idx,val in enumerate(lst):
```

good habit : don't modify loop variable

Display

```
print("v=",3,"cm :",x,",",y+4)
```

items to display : literal values, variables, expressions

`print` options:

- `sep=" "` — items separator, default space
- `end="\n"` — end of print, default new line
- `file=sys.stdout` — print to file, default standard output

Input

```
s = input("Instructions:")
```

`input` always returns a **string**, convert it to required type (cf. boxed *Conversions* on the other side).

Generic Operations on Containers

`len(c)` → items count

`min(c)` `max(c)` `sum(c)`

`sorted(c)` → `list` sorted *copy*

`val in c` → boolean, membership operator `in` (absence `not in`)

`enumerate(c)` → *iterator* on (index, value)

`zip(c1,c2...)` → *iterator* on tuples containing c_i items at same index

`all(c)` → `True` if **all** `c` items evaluated to true, else `False`

`any(c)` → `True` if **at least one** item of `c` evaluated true, else `False`

Note: For dictionaries and sets, these operations use ***keys****.*

Specific to ***ordered sequences containers*** *(lists, tuples, strings, bytes...)*

`reversed(c)` → *inversed iterator* `c*5`→ duplicate `c+c2`→ concatenate

`c.index(val)` → *position* `c.count(val)` → *events count*

```
import copy
copy.copy(c) → shallow copy of container
copy.deepcopy(c) → deep copy of container
```

Integer Sequences

`range([start,] end [,step])`

`start` default 0, `end` not included in sequence, `step` signed, default 1

`range(5)` → 0 1 2 3 4 `range(2,12,3)` → 2 5 8 11

`range(3,8)` → 3 4 5 6 7 `range(20,5,-5)` → 20 15 10

`range(len(seq))` → *sequence of index of values in seq*

range provides an immutable sequence of int constructed as needed

Operations on Lists

modify original list

`lst.append(val)`	add item at end
`lst.extend(seq)`	add sequence of items at end
`lst.insert(idx, val)`	insert item at index
`lst.remove(val)`	remove first item with value *val*
`lst.pop([idx])` → *value*	remove & return item at index *idx* (default last)
`lst.sort()` `lst.reverse()`	sort / reverse liste *in place*

Function Definition

function name (identifier)

named parameters

```
def fct(x,y,z):
    """documentation"""
    # statements block, res computation, etc.
    return res
```

result value of the call, if no computed result to return: `return None`

parameters and all variables of this block exist only *in* the block and *during* the function call (think of a "black box")

Advanced: `def fct(x,y,z,*args,a=3,b=5,**kwargs):`

**args variable positional arguments (→`tuple`), default values,*
***kwargs variable named arguments (→`dict`)*

Operations on Dictionaries

`d[key]=value`	`d.clear()`
`d[key]` → *value*	`del d[key]`

`d.update(d2)` — update/add associations

`d.keys()` `d.values()` `d.items()` → *iterable views on keys/values/associations*

`d.pop(key[,default])` → *value*

`d.popitem()` → *(key,value)*

`d.get(key[,default])` → *value*

`d.setdefault(key[,default])` → *value*

Operations on Sets

Operators:

- `|` → union (vertical bar char)
- `&` → intersection
- `-` `^` → difference/symmetric diff.
- `<` `<=` `>` `>=` → inclusion relations

Operators also exist as methods.

`s.update(s2)` `s.copy()`

`s.add(key)` `s.remove(key)`

`s.discard(key)` `s.clear()`

`s.pop()`

Function Call

```
r = fct(3,i+2,2*i)
```

storage/use of returned value

one argument per parameter

this is the use of function name *with parentheses* which does the call

Advanced:
**sequence*
***dict*

Operations on Strings

`s.startswith(prefix[,start[,end]])` `s.endswith(suffix[,start[,end]])` `s.strip([chars])`

`s.count(sub[,start[,end]])` `s.partition(sep)` → *(before,sep,after)*

`s.index(sub[,start[,end]])` `s.find(sub[,start[,end]])`

`s.is...()` *tests on chars categories (ex.* `s.isalpha()`*)*

`s.upper()` `s.lower()` `s.title()` `s.swapcase()`

`s.casefold()` `s.capitalize()` `s.center([width,fill])`

`s.ljust([width,fill])` `s.rjust([width,fill])` `s.zfill([width])`

`s.encode(encoding)` `s.split([sep])` `s.join(seq)`

Files

storing data on disk, and reading it back

```
f = open("file.txt","w",encoding="utf8")
```

file **variable** for operations

name of file on disk (+path...)

opening **mode**
- `'r'` read
- `'w'` write
- `'a'` append
- ...`'+'` `'x'` `'b'` `'t'`

encoding of chars for *text files*: `utf8` `ascii` `latin1` ...

cf. modules `os`, `os.path` and `pathlib`

writing

`f.write("coucou")`

`f.writelines(list of lines)`

reading

read empty string if end of file

`f.read([n])` → *next chars* — *if n not specified, read up to end !*

`f.readline()` → *next line*

`f.readline()` → `list` *of next lines*

text mode `t` *by default (read/write* `str`*), possible binary mode* `b` *(read/write* `bytes`*). Convert from/to required type !*

`f.close()` — dont forget to **close the file** after use !

`f.flush()` write cache `f.truncate([size])` resize

reading/writing progress sequentially in the file, modifiable with:

`f.tell()`→*position* `f.seek(position[,origin])`

Very common: opening with a guarded block (automatic closing) and reading loop on lines of a text file:

```
with open(...) as f:
    for line in f :
        # processing of line
```

Formatting

formating directives — values to format

```
"modele{} {} {}".format(x,y,r) → str
```

`"{selection : formatting ! conversion}"`

- **Selection** :
  `2`
  `nom`
  `0.nom`
  `4[key]`
  `0[2]`

Examples:

```
"{:+2.3f}".format(45.72793)
→'+45.728'
"{1:>10s}".format(8,"toto")
→'      toto'
"{x!r}".format(x="I'm")
→'"I\'m"'
```

- **Formatting** :

fill char *alignment* *sign* *mini width* . *precision~maxwidth* *type*

`< > ^ =` `+ - space` `0` at start for filling with 0

integer: `b` binary, `c` char, `d` decimal (default), `o` octal, `x` or `X` hexa...
float: `e` or `E` exponential, `f` or `F` fixed point, `g` or `G` appropriate (default),
string: `s` ... `%` percent

- **Conversion** : `s` (readable text) or `r` (literal representation)

Ficheros requeridos

prueba.py

```python
# PROBANDO la clase CheckoutQueue:
from supermarket import *

cola = CheckoutQueue()
print(cola) # ➡️None⬅️
cola.enqueue(Client("Ana", 35))
cola.enqueue(Client("Luis", 50))
cola.enqueue(Client("Marta", 25))
print(cola) # ➡️[Ana: 35]->[Luis: 50]->[Marta: 25]⬅️->None
print(cola.dequeue()) # Ana: 35
print(cola.dequeue()) # Luis: 50
print(cola.daily_sales) # 85
print(cola) # ➡️[Marta: 25]⬅️->None

cola.enqueue(Client("José", 40))
cola.enqueue(Client("Laura", 60))
print(cola) # ➡️[Marta: 25]->[José: 40]->[Laura: 60]⬅️->None
nueva_cola = cola.split()
print(cola) # ➡️[Marta: 25]->[Laura: 60]⬅️->None
print(nueva_cola) # ➡️[José: 40]⬅️->None

```

supermarket.py

```python
from containerbase import ContainerBase


class Client:

    def __init__(self, name, total_cost):
        self.name = name
        self.total_cost = total_cost

    def __str__(self):
        return f"{self.name}: {self.total_cost}"


class CheckoutQueue(ContainerBase):
    class Node:
        def __init__(self, value, next=None):
            self.value = value  # El valor en el nodo.
            self.next = next    # El enlace al siguiente nodo.

    def __init__(self):
        self.first = None   # El enlace al primer nodo de la lista.
        self.last = None   # El enlace al último nodo de la lista.

```

VPL